

Lab 7

Lab 6 added a controllable player and a camera that followed them — but at the cost of architectural shortcuts. The renderer was a singleton, IDs were assigned by hand, the camera could pan past the edge of the world, and several responsibilities were tangled across classes. Lab 7 is largely a refactor: the singleton goes away, the render loop moves to where it belongs, the camera respects world boundaries, and IDs assign themselves.

GameCamera — bounded panning

The camera class is rewritten with two important additions: it now requires its dimensions at construction time, and it knows about the limits of the world.

```
public class GameCamera
{
    private int _x;
    private int _y;
    private Rectangle<int> _worldBounds = new();

    public int X => _x;
    public int Y => _y;

    public readonly int Width;
    public readonly int Height;

    public GameCamera(int width, int height)
    {
        Width = width;
        Height = height;
    }
}
```

`X` and `Y` are now read-only from the outside — callers can no longer assign to them directly. Movement happens through the `LookAt` method, which is the second new addition. Before that can be used, the camera needs to know the size of the world it lives in:

```
public void SetWorldBounds(Rectangle<int> bounds)
{
    var marginLeft = Width / 2;
    var marginTop = Height / 2;

    if (marginLeft * 2 > bounds.Size.X)
    {
        marginLeft = 48;
    }

    if (marginTop * 2 > bounds.Size.Y)
    {
        marginTop = 48;
    }
}
```

```

    }

    _worldBounds = new Rectangle<int>(marginLeft, marginTop, bounds.Size.X -
    marginLeft * 2,
        bounds.Size.Y - marginTop * 2);
    _x = marginLeft;
    _y = marginTop;
}

```

`SetWorldBounds` computes the region where the camera position is allowed to sit. Since the camera centers itself, the leftmost valid `x` is half a screen-width from the left edge of the world, and the rightmost is half a screen-width from the right edge — otherwise the player would see past the edge of the map. The fallback 48 handles the case where the world is smaller than the camera view.

`LookAt` then clamps the camera to those bounds:

```

public void LookAt(int x, int y)
{
    if (_worldBounds.Contains(new Vector2D<int>(_x, y)))
    {
        _y = y;
    }
    if (_worldBounds.Contains(new Vector2D<int>(x, _y)))
    {
        _x = x;
    }
}

```

Each axis is checked independently, which is a subtle but useful detail — moving the player off the right edge still allows vertical movement to update normally, rather than freezing both axes. The result is that the player can walk to a corner without the camera getting "stuck" before they reach it.

The two coordinate transformation methods (`ToScreenCoordinates`, `ToWorldCoordinates`) are unchanged.

GameRenderer — singleton removed, role narrowed

`GameRenderer` undergoes the largest restructuring. The partial class split from Lab 4 is gone, and so is the `_instance` static field that made `LoadTexture` callable from anywhere. The renderer is now a plain object that gets passed around explicitly:

```

public unsafe class GameRenderer
{
    private Sdl _sdl;
    private Renderer* _renderer;
    private GameWindow _window;
    private GameCamera _camera;
}

```

```

private Dictionary<int, IntPtr> _texturePointers = new();
private Dictionary<int, TextureData> _textureData = new();
private int _textureId;

public GameRenderer(Sdl sdl, GameWindow window)
{
    _sdl = sdl;

    _renderer = (Renderer*)window.CreateRenderer();
    _sdl.SetRenderDrawBlendMode(_renderer, BlendMode.Blend);

    _window = window;
    var windowSize = window.Size;
    _camera = new GameCamera(windowSize.Width, windowSize.Height);
}

```

A few things to note. The constructor no longer takes a `GameLogic` reference — the dependency direction has flipped, with `GameLogic` now holding the renderer instead. The camera is constructed with the window's actual dimensions baked in.

`LoadTexture` becomes an instance method:

```

public int LoadTexture(string fileName, out TextureData textureInfo)
{
    using (var fStream = new FileStream(fileName, FileMode.Open))
    {
        var image = Image.Load<Rgba32>(fStream);
        textureInfo = new TextureData()
        {
            Width = image.Width,
            Height = image.Height
        };
        var imageRAWData = new byte[textureInfo.Width * textureInfo.Height *
4];
        image.CopyPixelDataTo(imageRAWData.AsSpan());
        fixed (byte* data = imageRAWData)
        {
            var imageSurface = _sdl.CreateRGBSurfaceWithFormatFrom(data,
textureInfo.Width,
            textureInfo.Height, 8, textureInfo.Width * 4,
(uint)PixelFormatEnum.Rgba32);
            if (imageSurface == null)
            {
                throw new Exception("Failed to create surface from image
data.");
            }

            var imageTexture = _sdl.CreateTextureFromSurface(_renderer,
imageSurface);
            if (imageTexture == null)
            {

```

```

        _sdl.FreeSurface(imageSurface);
        throw new Exception("Failed to create texture from surface.");
    }

    _sdl.FreeSurface(imageSurface);

    _textureData[_textureId] = textureInfo;
    _texturePointers[_textureId] = (IntPtr)imageTexture;
}

return _textureId++;
}

```

The body is essentially the same as before, but it now operates on the instance's own `_sdl`, `_renderer`, and dictionaries rather than reaching through `_instance`!. Error handling is also stricter — `null` from SDL throws an exception with a useful message instead of returning `-1`.

`RenderGameObject` is gone entirely. Everything now goes through a single, more capable `RenderTexture`:

```

public void RenderTexture(int textureId, Rectangle<int> src, Rectangle<int>
dst,
    RendererFlip flip = RendererFlip.None, double angle = 0.0, Point center =
default)
{
    if (_texturePointers.TryGetValue(textureId, out var imageTexture))
    {
        var translatedDst = _camera.ToScreenCoordinates(dst);
        _sdl.RenderCopyEx(_renderer, (Texture*)imageTexture, in src,
            in translatedDst,
            angle,
            in center, flip);
    }
}

```

The new optional parameters (`flip`, `angle`, `center`) absorb the rotation features that previously lived in `RenderGameObject`. Using default values means existing call sites — terrain tiles, player sprite — don't need to change. SDL's `RenderCopyEx` handles both the simple case (no rotation, no flip) and the rich case identically.

Finally, the renderer exposes the low-level frame operations as named methods:

```

public void SetDrawColor(byte r, byte g, byte b, byte a) =>
    _sdl.SetRenderDrawColor(_renderer, r, g, b, a);
public void ClearScreen() => _sdl.RenderClear(_renderer);
public void PresentFrame() => _sdl.RenderPresent(_renderer);
public void SetWorldBounds(Rectangle<int> bounds) =>

```

```

_camera.SetWorldBounds(bounds);
public void CameraLookAt(int x, int y) => _camera.LookAt(x, y);
public Vector2D<int> ToWorldCoordinates(int x, int y) =>
_camera.ToWorldCoordinates(new Vector2D<int>(x, y));

```

These are thin pass-throughs, but they let `GameLogic` orchestrate a frame without having to know that an SDL renderer or a camera object exists underneath. The `Render` method that previously drove the entire frame is gone — frame composition has moved up to `GameLogic`.

GameLogic — owning the render loop

`GameLogic` now holds the renderer and is responsible for assembling each frame:

```

private readonly GameRenderer _renderer;

public GameLogic(GameRenderer renderer)
{
    _renderer = renderer;
}

```

This inversion is the architectural heart of the refactor. Previously, `GameRenderer.Render()` would call back into `_gameLogic.RenderTerrain()` and `_gameLogic.RenderAllObjects()` — a two-way dependency that made it hard to reason about who controlled what. Now the dependency only goes one way: `GameLogic` calls into `GameRenderer`.

`InitializeGame` is updated for the new API and takes on a new responsibility — telling the renderer how big the world is:

```

if (level.Width == null || level.Height == null)
{
    throw new Exception("Invalid level dimensions");
}

if (level.TileWidth == null || level.TileHeight == null)
{
    throw new Exception("Invalid tile dimensions");
}

_renderer.SetWorldBounds(new Rectangle<int>(0, 0, level.Width.Value *
level.TileWidth.Value,
level.Height.Value * level.TileHeight.Value));

```

The world bounds are computed by multiplying the level's tile-grid dimensions by tile size — so for the 60×40 grid of 16×16 tiles from Lab 5, the world is 960×640 pixels.

A new `RenderFrame` method is the new center of gravity for drawing:

```

public void RenderFrame()
{
    var currentTime = DateTimeOffset.Now;
    var msSinceLastFrame = (currentTime - _lastUpdate).TotalMilliseconds;
    _lastUpdate = currentTime;

    _renderer.SetDrawColor(0, 0, 0, 255);
    _renderer.ClearScreen();

    _renderer.CameraLookAt(_player!.X, _player!.Y);

    RenderTerrain();
    RenderAllObjects(msSinceLastFrame);

    _renderer.PresentFrame();
}

```

This is now the canonical "what happens during a frame" sequence: measure elapsed time, clear to black, point the camera at the player, draw the terrain, draw the objects, flip the buffer. The frame timing logic (using `DateTimeOffset` deltas) that previously lived inside `GameRenderer` has moved here too.

Per-frame elapsed time is now a `double` (milliseconds with fractional precision) rather than an `int`, and that change ripples through the object system:

```

public void RenderAllObjects(double msSinceLastFrame)
{
    // ...
    if (gameObject.Update(msSinceLastFrame))
    // ...
}

```

`AddBomb` takes screen coordinates and does the world conversion itself — keeping `InputLogic` pleasantly ignorant of the camera:

```

public void AddBomb(int screenX, int screenY)
{
    var worldCoords = _renderer.ToWorldCoordinates(screenX, screenY);
    AnimatedGameObject bomb =
        new AnimatedGameObject(Path.Combine("Assets", "BombExploding.png"),
    _renderer, 2, 13, 13, 1,
        worldCoords.X, worldCoords.Y);
    _gameObjects.Add(bomb.Id, bomb);
    ++_bombIds;
}

```

Note that the bomb constructor no longer takes an ID — that's now generated automatically. And the

manually-tracked `_bombIds` counter is incremented but no longer used for anything meaningful; the counter survives the refactor as a vestige.

The `GetPlayerPosition` method is removed entirely. The renderer used to call it from inside its own render loop to position the camera — now `GameLogic` just calls `_renderer.CameraLookAt(_player!.X, _player!.Y)` directly from its frame method.

GameObject — automatic ID assignment

`GameObject` becomes much simpler — IDs are now assigned automatically using a thread-safe counter:

```
public class GameObject
{
    public int Id { get; private set; }

    private static int _nextId = -1;

    public GameObject()
    {
        Id = Interlocked.Increment(ref _nextId);
    }
}
```

`Interlocked.Increment` atomically increments the counter and returns the new value, which makes ID generation safe even if game objects are ever created from multiple threads. The starting value of `-1` means the first object gets ID `0`. The whole "give every object an explicit ID" ritual that ran through every constructor since Lab 4 is now gone — subclasses just call `base()`.

Both `RenderableGameObject` and `AnimatedGameObject` are updated to take a `GameRenderer` reference instead of an ID, since they need the renderer to load their textures (the singleton workaround from Lab 4 is no longer available):

```
public RenderableGameObject(string fileName, GameRenderer renderer)
{
    TextureId = renderer.LoadTexture(fileName, out var textureData);
    // ...
}
```

`RenderableGameObject.Render` is also simplified. Where it previously delegated to `renderer.RenderGameObject(this)`, it now calls the unified `RenderTexture` directly:

```
public virtual void Render(GameRenderer renderer)
{
    renderer.RenderTexture(TextureId, TextureSource, TextureDestination);
}
```

`PlayerObject` follows the same pattern — it now takes a renderer in its constructor instead of an ID and gets its ID from the base class automatically.

AnimatedGameObject — fractional time

The animation class now uses `double` to track elapsed time, matching the new precision passed in by `RenderAllObjects`:

```
private double _timeSinceAnimationStart = 0;

public override bool Update(double msSinceLastFrame)
{
    _timeSinceAnimationStart += msSinceLastFrame;

    var currentFrame = _timeSinceAnimationStart / _timePerFrame;

    if (_timeSinceAnimationStart > _durationInSeconds * 1000) return false;

    _currentRow = (int)(currentFrame / _numberOfColumns);
    _currentColumn = (int)(currentFrame % _numberOfColumns);
    // ...
}
```

The two `(int)` casts at the end keep the row/column indices integral, but the underlying division happens in floating point. With short frames (~13ms) the fractional accumulation matters — small per-frame increments would otherwise be rounded away to zero with integer arithmetic.

InputLogic — simplified bomb call

Now that `GameLogic.AddBomb` does its own world coordinate conversion, `InputLogic` no longer needs to:

```
if (mouseButtonStates[(byte)MouseButton.Primary] == 1)
{
    _gameLogic.AddBomb(mouseX, mouseY);
}
```

This is a small change but reflects the broader cleanup: `InputLogic` no longer needs to know about the camera at all.

GameWindow — smaller window

The window dimensions shrink from 800×800 to 640×400:

```
_window = (IntPtr)sdl.CreateWindow(
    "The Adventure", Sdl.WindowposUndefined, Sdl.WindowposUndefined, 640, 400,
    (uint)WindowFlags.Resizable | (uint)WindowFlags.AllowHighdpi
```



```
);
```

640×400 is smaller than the 960×640 world (60×40 tiles at 16 pixels each), which is the case the bounded camera was designed for — there is now meaningful room for the world to be larger than what is visible, and the camera will scroll to keep the player centered.

Program.cs — cleaner main loop

The construction order changes to match the new dependencies — renderer first, then logic that holds the renderer:

```
var gameWindow = new GameWindow(sdl);
var gameRenderer = new GameRenderer(sdl, gameWindow);
var gameLogic = new GameLogic(gameRenderer);
var inputLogic = new InputLogic(sdl, gameLogic);
```

The main loop sheds the leftover frame counter and **Stopwatch** from earlier labs, and drops the per-frame sleep from 50ms to 13ms (roughly 75 FPS):

```
while (!quit)
{
    quit = inputLogic.ProcessInput();
    if (quit) break;

    gameLogic.ProcessFrame();
    gameLogic.RenderFrame();

    Thread.Sleep(13);
}
```

`gameRenderer.Render()` no longer exists — `gameLogic.RenderFrame()` is the entry point. The shorter sleep makes the game noticeably smoother, which combined with the floating-point time tracking in **AnimatedGameObject** produces visibly cleaner explosion animations.

Summary

By the end of Lab 7, the codebase looks like a small game engine rather than a sequence of demos. The renderer has a single, narrow responsibility: drawing pixels and translating coordinates. The game logic owns the frame loop and orchestrates terrain, objects, and the player. Game objects assign their own IDs and can be created without manual bookkeeping. The camera respects the edges of the world. The dependencies between components form a clear graph instead of the singleton-mediated tangle from earlier labs.